Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Programovanie v Pythone
Semestrálny projekt – technická dokumentácia

# DROPZONE – sledovanie cien PC hier

Autor: **Ivan Nerodyk**
Akademický rok: 2026/2027
Repozitár: https://git.kpi.fei.tuke.sk/python/2026/ivan-nerodyk
GitHub (pracovná kópia a zber cien):
https://github.com/titoptitop733-source/Proverca-igor

# Obsah

**1 Úvod . . . 3**
**2 Požiadavky a funkcionalita . . . 3**
2.1 Funkčné požiadavky . . . 3
2.2 Nefunkčné požiadavky . . . 3
2.3 Používateľské scenáre . . . 4
2.4 MVP a finálna verzia . . . 4
**3 Návrh riešenia . . . 4**
3.1 Princípy návrhu . . . 4
3.2 Tok dát pri aktualizácii cien . . . 5
3.3 Zber cien v cloude . . . 5
**4 Použité technológie . . . 6**
**5 Implementácia . . . 7**
5.1 Moduly . . . 7
5.2 Dátový model . . . 7
5.3 Integrácia s CheapShark API . . . 8
5.4 Analýza a odporúčanie . . . 8
5.5 REST API . . . 8
5.6 Webové rozhranie . . . 9
5.7 Desktopová aplikácia a meny . . . 9
5.8 Synchronizácia s cloudom (sync.py) . . . 9
**6 Testovanie a robustnosť . . . 9**
6.1 Chybové situácie . . . 10
6.2 Kvalita kódu . . . 10
6.3 Manuálne overenie . . . 10
**7 Bezpečnosť . . . 10**
**8 Použitie programu . . . 11**
8.1 Inštalácia a spustenie . . . 11
8.2 Webová aplikácia . . . 11
8.3 Desktopová aplikácia a CLI . . . 13
8.4 Zber cien v cloude . . . 14
**9 Čo som sa naučil . . . 14**
**10 Obmedzenia a možné rozšírenia . . . 14**
10.1 Obmedzenia . . . 14
10.2 Možné rozšírenia . . . 15
**11 Zdroje a použité nástroje . . . 15**
11.1 Použitie AI nástrojov . . . 15

# 1 Úvod

Ceny digitálnych hier sa často menia a v jednotlivých obchodoch (Steam, GOG, Humble Store, Epic Games Store a ďalšie) sa líšia. Kto chce hru kúpiť výhodne, musí ceny sledovať ručne vo viacerých obchodoch. Ľahko tak premešká zľavu, alebo naopak kúpi hru tesne pred väčším výpredajom.

Projekt **DROPZONE** tento problém rieši. Aplikácia pravidelne zaznamenáva ceny sledovaných hier zo všetkých obchodov do lokálnej databázy, zobrazuje vývoj ceny, porovnáva obchody a na základe zaznamenanej histórie odporúča, či je aktuálna cena výhodná (*kúpiť teraz*), alebo sa oplatí počkať. Používateľ si môže nastaviť cieľovú cenu a aplikácia ho upozorní, keď ju cena dosiahne.

Tému som si zvolil, pretože hry na PC kupujem pravidelne a porovnávanie cien je pre mňa reálny problém. Téma zároveň prirodzene spája viacero oblastí predmetu: prácu s externými API, databázu, analýzu dát, webový backend a používateľské rozhranie. Každá z týchto oblastí rieši skutočnú časť problému, nie je pridaná len formálne.

Hlavným cieľom bolo vytvoriť aplikáciu, ktorá pracuje s **reálnymi dátami** (nie s ukážkovými), dá sa spustiť na inom počítači iba podľa README, bez API kľúčov, a ktorej odporúčania sú vysvetliteľné a overiteľné – aplikácia vždy uvádza dôvody, prečo odporúča kúpiť alebo počkať. Ceny sa zbierajú automaticky aj v cloude (GitHub Actions), takže história vzniká aj vtedy, keď je počítač vypnutý.

# 2 Požiadavky a funkcionalita

## 2.1 Funkčné požiadavky

- vyhľadanie ľubovoľnej PC hry podľa názvu a jej pridanie do zoznamu sledovaných hier (wishlist),
- pravidelný záznam cien zo všetkých obchodov (tzv. snímky) do databázy,
- história ceny, štatistiky (historické minimum, priemer, maximum, frekvencia zliav) a graf,
- odporúčanie *BUY NOW / GOOD DEAL / WAIT* s vysvetlením dôvodov,
- cieľová cena so stavom *dosiahnutá / takmer / čaká sa* a upozornenie pri jej dosiahnutí,
- prehľad (dashboard) s potenciálnou úsporou, poklesmi cien a históriou udalostí,
- stránka *Discover* s aktuálnymi zľavami, filtrami (obchod, cena, zľava) a triedením,
- zobrazenie cien v USD, EUR alebo GBP podľa aktuálnych kurzov ECB,
- automatický zber cien v cloude každých 6 hodín (GitHub Actions) a jeho zlúčenie s lokálnou databázou,
- tmavá a svetlá téma (alebo podľa systému),
- tri rozhrania nad spoločnou logikou: webová aplikácia, desktopová aplikácia a CLI.

## 2.2 Nefunkčné požiadavky

- spustiteľnosť podľa README bez API kľúčov a bez ručne vytvorenej databázy,
- odolnosť voči výpadku externých služieb – aplikácia nespadne, zobrazí zrozumiteľnú chybu,

- server dostupný iba z lokálneho počítača (127.0.0.1),
- automatické testy bez prístupu na internet,
- konfigurácia oddelená od kódu (súbor `.env`).

### 2.3 Používateľské scenáre

| Scenár | Priebeh |
|---|---|
| S1 Sledovanie novej hry | Používateľ klikne na *Track a game*, vyhľadá hru a pridá ju. Aplikácia uloží prvú snímku cien a hru zobrazí v prehľade. |
| S2 Rozhodnutie o kúpe | Otvorí detail hry, pozrie graf, porovnanie obchodov a kartu *Buy or wait?* s dôvodmi odporúčania. |
| S3 Hľadanie zľavy | Na stránke Discover nastaví maximálnu cenu a minimálnu zľavu, výhodnú hru začne sledovať jedným kliknutím. |
| S4 Čakanie na cenu | Nastaví cieľovú cenu. Pri ďalšej kontrole cien, keď cena klesne pod cieľ, aplikácia zobrazí upozornenie. |

Tabuľka 1: Hlavné používateľské scenáre

### 2.4 MVP a finálna verzia

| Oblasť | MVP (6. týždeň) | Finálna verzia |
|---|---|---|
| Rozhranie | CLI | webová aplikácia, desktop GUI, CLI |
| Dáta | wishlist, snímky cien | + obrázky hier, ponuky obchodov, kurzy mien |
| Analýza | – | štatistiky (pandas), odporúčanie, grafy, dashboard |
| Externé API | CheapShark (vyhľadávanie, ceny) | + CheapShark deals, Frankfurter (kurzy) |
| Automatizácia | ručné spustenie príkazu | GitHub Actions každých 6 h, synchronizácia |
| Kvalita | 20 testov | 120 testov, ruff, mypy |

Tabuľka 2: Porovnanie MVP a finálnej verzie

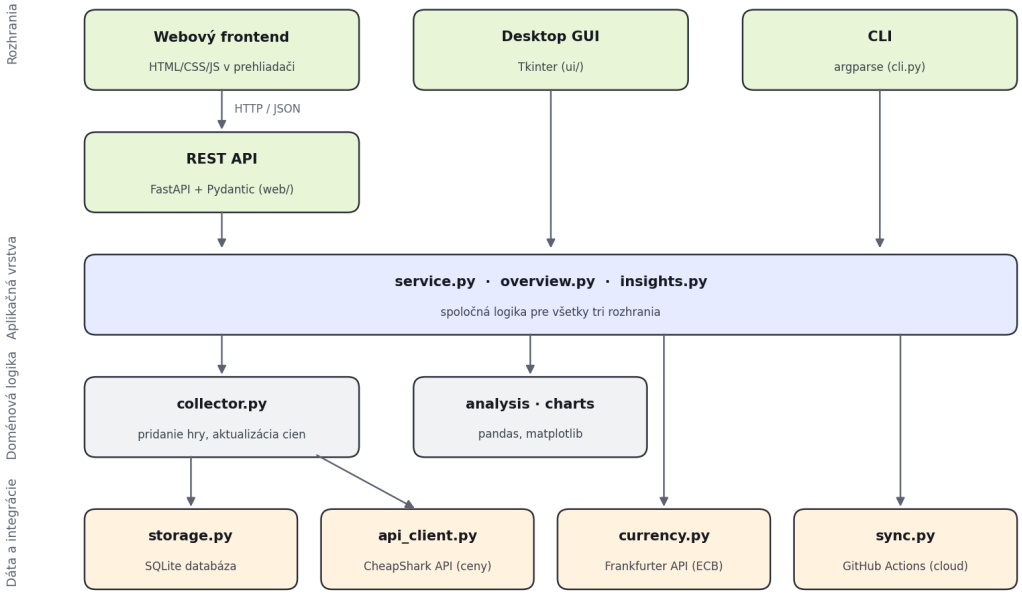
## 3 Návrh riešenia

Aplikácia je rozdelená do vrstiev. Rozhrania (web, desktop, CLI) neobsahujú aplikačnú logiku – iba zobrazujú dáta a volajú spoločnú aplikačnú vrstvu. Vďaka tomu sa rovnaký výpočet odporúčania alebo štatistík používa vo všetkých troch rozhraniach a dá sa testovať bez grafického rozhrania. Celkové usporiadanie ukazuje obrázok 1.

### 3.1 Princípy návrhu

- **Oddelenie rozhrania od logiky** – moduly *analysis*, *collector* a *insights* nepoznajú Tkinter ani FastAPI.
- **Vkladanie závislostí** – `create_app(service, rates)` dostane služby ako parametre, takže testy môžu použiť falošný zdroj dát bez siete.
- **Rozhranie zdroja dát** – trieda `GameDataSource` (typing.Protocol) opisuje, čo musí zdroj vedieť. Skutočný klient aj testovací fake ju spĺňajú.
- **Krátke databázové spojenia** – každá operácia v *service.py* otvorí vlastné spojenie, pretože spojenie sqlite3 sa nesmie zdieľať medzi vláknami (web server a GUI

používajú vlákna).




Obrázok 1: Architektúra aplikácie (šípky ukazujú smer volaní)

## 3.2 Tok dát pri aktualizácii cien

1. Rozhranie zavolá `TrackerService.update_prices()`.
2. Pre každú sledovanú hru sa overí, či od poslednej snímky uplynul nastavený interval (6 h).
3. Klient CheapShark stiahne aktuálne ponuky všetkých obchodov a historické minimum.
4. Ponuky sa uložia ako jedna snímka do tabuľky *price_history*.
5. Ak najlepšia cena dosiahla cieľovú cenu, vznikne upozornenie (PriceAlert).
6. Chyba pri jednej hre nezastaví aktualizáciu ostatných – zapíše sa do výsledného reportu.

## 3.3 Zber cien v cloude

Ceny sa zaznamenávajú len vtedy, keď aplikácia beží. Aby história vznikala aj bez zapnutého počítača, kontroluje ceny aj plánovaný workflow GitHub Actions. Každý údaj má práve jedného vlastníka, preto medzi počítačom a cloudom nevznikajú konflikty:

| Krok | Kde | Čo sa deje |
|---|---|---|
| 1 | počítač | `game-tracker sync --push` zapíše sledované hry a cieľové ceny do *data/wishlist.json* a odošle súbor do repozitára |
| 2 | GitHub Actions | každých 6 h: `game-tracker collect` prečíta wishlist.json, obnoví databázu z vetvy *data* a skontroluje ceny |
| 3 | GitHub | databáza sa uloží ako jediný commit vo vetve *data*, repozitár preto nerastie |
| 4 | počítač | `game-tracker web` pri štarte stiahne databázu a zlúči chýbajúce snímky |

Tabuľka 3: Tok dát medzi počítačom a cloudom

# 4 Použité technológie

Projekt kombinuje šesť technologických oblastí predmetu (ďalej testovanie a kontrolu kvality). Každá rieši konkrétnu časť problému, ako ukazuje nasledujúca tabuľka.

| Oblasť | Knižnica | Účel v projekte | Prečo |
|---|---|---|---|
| HTTP a externé API | httpx | klient pre CheapShark a Frankfurter | timeouty, jednoduché mapovanie chýb, MockTransport na testy bez siete |
| Databázy | sqlite3 | wishlist, obchody, história cien | súčasť štandardnej knižnice, netreba server; podpora window funkcií |
| Data Science | pandas, matplotlib | štatistiky, odporúčanie, grafy | rýchla práca s časovými radmi; grafy pre CLI aj Tkinter |
| Web backend | FastAPI, Pydantic, uvicorn | REST API pre webové rozhranie | validácia vstupov, automatická dokumentácia OpenAPI (/docs) |
| Používateľské rozhranie | HTML/CSS/JS, Tkinter | webová a desktopová aplikácia | web bez build kroku (netreba Node.js); Tkinter je súčasť Pythonu |
| Skriptovanie a automatizácia | argparse, pathlib, subprocess, GitHub Actions | CLI, konfigurácia, git push wishlistu, plánovaný zber cien | konfigurácia mimo kódu; zber cien bez zapnutého počítača |
| Kvalita kódu | pytest, ruff, mypy | 120 testov, štýl, kontrola typov | fixtures, tmp_path, monkeypatch; type hints overené nástrojom |

Tabuľka 4: Technologické oblasti a ich použitie

Závislosti sú uvedené v súbore `pyproject.toml` a inštalujú sa jedným príkazom `pip install -e ".[dev]"`.

# 5 Implementácia

## 5.1 Moduly

| Modul | Zodpovednosť |
|---|---|
| api_client.py | HTTP klient CheapShark: vyhľadávanie, detail hry, zľavy, obchody; prevod JSON na dátové triedy |
| currency.py | kurzy mien z Frankfurter API, vyrovnávacia pamäť na 12 h, záložné správanie |
| models.py | nemenné dátové triedy (dataclass): Game, Deal, PricePoint, DealListing, ... |
| storage.py | repozitár nad SQLite, parametrizované dotazy, automatické migrácie |
| collector.py | pridanie hry, aktualizácia cien, interval snímok, upozornenia |
| analysis.py | štatistiky cez pandas a pravidlá odporúčania |
| insights.py | stav cieľovej ceny, udalosti (pokles, nové minimum), súhrn dashboardu |
| overview.py, service.py | spojenie dát a analýzy; fasáda pre GUI a web, bezpečná pre vlákna |
| sync.py | zber v cloude: wishlist.json, stiahnutie a zlúčenie databázy, git push |
| web/app.py, schemas.py | FastAPI aplikácia, endpointy, Pydantic modely požiadaviek a odpovedí |
| web/static/ | frontend: index.html, css/app.css, js/ (ES moduly, vlastné SVG grafy) |
| ui/ | Tkinter aplikácia, beh HTTP požiadaviek mimo hlavného vlákna |
| cli.py, config.py | príkazový riadok, načítanie a validácia konfigurácie |

Tabuľka 5: Hlavné moduly (src/game_tracker)

## 5.2 Dátový model

Databáza má tri tabuľky. Každé spustenie aktualizácie vytvorí pre hru jednu *snímku* – skupinu riadkov v *price_history* s rovnakým časom *recorded_at*, jeden riadok pre každý obchod.

| Tabuľka | Stĺpce | Poznámka |
|---|---|---|
| games | game_id (PK), title, target_price, added_at, cheapest_ever_price, steam_app_id, thumb_url | CHECK: target_price ≥ 0 |
| stores | store_id (PK), name, is_active | zoznam obchodov z API |
| price_history | id (PK), game_id (FK, ON DELETE CASCADE), store_id, price, retail_price, savings_percent, recorded_at | index (game_id, recorded_at) |

Tabuľka 6: Databázová schéma

Nové stĺpce pridané počas vývoja (napr. *steam_app_id*) sa do existujúcej databázy doplnia automaticky pri štarte (`Storage._migrate`), takže používateľ nestratí nazbierané dáta. Najlepšiu cenu každej snímky vyberá SQL dotaz s window funkciou:

```
SELECT ... FROM (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY recorded_at ORDER BY price, store_id) AS rank_in_snapshot
    FROM price_history WHERE game_id = ?
) h WHERE h.rank_in_snapshot = 1 ORDER BY h.recorded_at
```

## 5.3 Integrácia s CheapShark API

Používajú sa endpointy */games* (vyhľadávanie a detail), */deals* (aktuálne zľavy) a */stores*. API vracia čísla ako reťazce, preto každá hodnota prechádza validáciou; neplatná odpoveď vyvolá výnimku `ApiError` namiesto pádu programu. Timeout, kód 429 (limit požiadaviek), chyby 5xx a neplatný JSON majú vlastné zrozumiteľné správy. URL obrázkov sa akceptujú iba so schémou *https://*.

## 5.4 Analýza a odporúčanie

Modul *analysis.py* prevedie najlepšie ceny jednotlivých snímok na DataFrame a vypočíta štatistiky. Historické minimum kombinuje vlastné záznamy s minimom, ktoré hlási CheapShark – pokrýva teda aj obdobie pred začiatkom sledovania. Odporúčanie sa vyhodnocuje podľa pravidiel v tomto poradí:

| Verdikt | Podmienka |
|---|---|
| BUY NOW | cena dosiahla cieľovú cenu, alebo je najviac 10 % nad historickým minimom |
| GOOD DEAL | zľava aspoň 50 %, alebo cena aspoň 15 % pod zaznamenaným priemerom |
| WAIT | hra už bola aspoň o 30 % lacnejšia, alebo momentálne nie je v zľave |
| NO DATA | zatiaľ neexistuje žiadna snímka ceny |

Tabuľka 7: Pravidlá odporúčania

Pri menej ako troch snímkach sa k odporúčaniu pridá upozornenie na nízku spoľahlivosť. Prahy sú konštanty na začiatku modulu. Pravidlá zámerne vysvetľujú minulé ceny a netvrdia, že predpovedajú budúce výpredaje.

## 5.5 REST API

| Metóda a cesta | Popis | Kódy |
|---|---|---|
| GET /api/dashboard | súhrn, najväčšia zľava, udalosti | 200 |
| GET /api/wishlist | sledované hry s cenou, stavom a odporúčaním | 200 |
| POST /api/wishlist | pridanie hry {game_id, target_price} | 201, 409, 422, 502 |
| DELETE /api/wishlist/{id} | zrušenie sledovania | 204, 404 |
| PUT /api/wishlist/{id}/target | nastavenie alebo zrušenie cieľovej ceny | 200, 404, 422 |
| GET /api/games/{id} | detail: štatistiky, história, ponuky obchodov | 200, 404 |
| POST /api/update | kontrola cien {force} | 200 |
| GET /api/search, /api/deals | vyhľadávanie hier, aktuálne zľavy s filtrami | 200, 422, 502 |
| GET /api/stores, /api/meta | obchody; verzia, kurzy mien, zdroje dát | 200 |

Tabuľka 8: Endpointy REST API (ceny sú v USD)

Chybu externého API mapuje handler na kód 502 s textom chyby, neplatný vstup odmietne Pydantic kódom 422. Frontend tieto správy zobrazuje používateľovi.

### 5.6 Webové rozhranie

Frontend je jednostránková aplikácia bez build kroku – súbory servuje priamo FastAPI. Kód je rozdelený do ES modulov: *api.js* (komunikácia so serverom), *format.js* (formátovanie a prevod mien), *charts.js* (vlastné SVG grafy s tooltipom), *ui.js* (modálne okná, notifikácie) a samostatné moduly pre každú stránku. Všetok text z API prechádza funkciou `esc()`, ktorá bráni vloženiu HTML (XSS). Pri prechode medzi stránkami kontroluje funkcia `active()`, či odpoveď ešte patrí k aktuálnej stránke, takže pomalá odpoveď neprepíše novú stránku. Preferencie (mena, rozloženie) sa ukladajú do localStorage.

### 5.7 Desktopová aplikácia a meny

Tkinter dovoľuje meniť okná iba z hlavného vlákna. HTTP požiadavky preto bežia v ThreadPoolExecutor a výsledok sa preberá periodicky cez `after()` (modul *ui/background.py*). Kurzy mien drží *currency.py* v pamäti 12 hodín; prístup chráni zámok, pretože web server obsluhuje požiadavky paralelne. Ak služba nie je dostupná, použijú sa posledné známe kurzy, prípadne iba USD.

### 5.8 Synchronizácia s cloudom (sync.py)

Workflow *.github/workflows/collect-prices.yml* beží každých 6 hodín (aj ručne cez tlačidlo *Run workflow*). Zoznam hier berie zo súboru *wishlist.json*, databázu obnoví z vetvy *data*, skontroluje ceny a databázu uloží späť ako jediný commit. Lokálna strana stiahne súbor *game_tracker.db* a funkcia `merge_remote_database` pridá iba snímky, ktoré lokálna databáza ešte nemá. Snímka sa identifikuje dvojicou (čas, obchod), takže zlúčenie je **idempotentné** – opakovaný beh nič nezmení a lokálne záznamy sa nikdy neprepíšu ani nezmažú. Stiahnutý súbor sa overuje (hlavička SQLite, limit veľkosti 50 MB) skôr, než sa použije.

## 6 Testovanie a robustnosť

Projekt obsahuje 120 automatických testov (pytest). Žiadny test nepoužíva internet: externé API nahrádza `httpx.MockTransport` alebo falošný zdroj dát, databáza vzniká v dočasnom adresári (`tmp_path`) a REST API sa testuje cez `TestClient` z FastAPI.

| Súbor | Počet | Čo overuje |
|---|---|---|
| test_api_client.py | 11 | parsovanie odpovedí, timeout, 429, 500, neplatné čísla, nebezpečné URL |
| test_storage.py | 12 | wishlist, snímky, najlepšia cena, kaskádové mazanie, migrácia, SQL injection |
| test_analysis.py | 10 | štatistiky a všetky vetvy odporúčania vrátane hry zadarmo |
| test_collector.py | 7 | interval snímok, --force, upozornenia, zlyhanie jednej hry |
| test_sync.py | 14 | wishlist.json, zlúčenie snímok (idempotencia), sťahovanie, neplatný súbor |
| test_web_api.py | 16 | endpointy, stavové kódy 201/204/404/409/422/502, validácia |
| test_insights.py | 5 | stav cieľa, udalosti, súhrn dashboardu |

| Súbor | Počet | Čo overuje |
|---|---|---|
| test_currency.py | 5 | kurzy, cache a jej expirácia, výpadok služby |
| ostatné (5 súborov) | 27 | konfigurácia, služba, grafy, formátovanie GUI, súbory frontendu |

Tabuľka 9: Prehľad automatických testov

## 6.1 Chybové situácie

| Situácia | Správanie aplikácie |
|---|---|
| CheapShark neodpovedá / timeout | zrozumiteľná správa (502 vo webe), aplikácia beží ďalej |
| prekročený limit požiadaviek (429) | správa „skúste neskôr“ |
| zlyhá aktualizácia jednej hry | ostatné hry sa aktualizujú, chyba je v reporte |
| nedostupné kurzy mien | posledné známe kurzy, inak zobrazenie v USD |
| neplatný vstup (záporná cena, text) | odmietnutie vo frontende aj v API (422) |
| pridanie už sledovanej hry | kód 409, frontend ukáže stav „Tracked“ |
| stará databáza z verzie 0.1 | automatická migrácia, údaje sa zachovajú |
| cloudová databáza chýba alebo je poškodená | zrozumiteľná správa, lokálne dáta sa nezmenia |
| chýbajúci obrázok hry | náhradný obrázok, potom názov hry |

Tabuľka 10: Ošetrenie chybových situácií

## 6.2 Kvalita kódu

Kvalitu kódu kontrolujú dva nástroje, nastavené v *pyproject.toml*. `ruff check src tests` (pravidlá E, F, W, I, B, UP) hlási *All checks passed*; `mypy src/game_tracker` hlási *no issues found in 27 source files*. Type hints sú použité v celom verejnom API projektu. Pri kontrole sa našlo napríklad nesprávne ošetrenie hodnoty None v súhrne dashboardu, ktoré sa opravilo. Rovnaké tri kontroly (ruff, mypy, pytest) bežia automaticky pri každom pushi v GitHub Actions (workflow *tests.yml*).

## 6.3 Manuálne overenie

Webové rozhranie bolo overené v prehliadači týmito scenármi: pridanie hry cez modálne okno aj globálne vyhľadávanie, zamedzenie duplicity, uloženie cieľovej ceny (aj s desatinnou čiarkou), odmietnutie neplatnej ceny, prepínanie rozsahu grafu a tooltip, filtre, triedenie a rozloženie wishlistu, odstránenie hry s potvrdením, filtre stránky Discover, zmena meny a jej uloženie po obnovení stránky, notifikácie, nastavenia a stránka neexistujúcej hry. Pri overovaní sa našli a opravili štyri chyby, napr. neuloženie cieľovej ceny pri vypnutom prepínači upozornení.

# 7 Bezpečnosť

- **Sieť** – web server počúva iba na 127.0.0.1, nie je dostupný z iných zariadení.
- **SQL** – všetky dotazy používajú parametre (?); test overuje názov hry s textom `'); DROP TABLE games;--`.

- **Vstupy** – Pydantic kontroluje typy a rozsahy (game_id > 0, cena 0 až 10 000, dĺžka hľadaného textu), CLI a GUI majú vlastnú validáciu.
- **XSS** – text z externých API sa vo frontende vždy escapuje; obrázky sa prijímajú iba z adries https.
- **Secrets** – projekt nepotrebuje API kľúče; konfigurácia je v .env, ktorý je v .gitignore. Voliteľný token pre súkromný repozitár (GT_SYNC_TOKEN) žije iba tam.
- **Cloud** – workflow používa iba vstavaný GITHUB_TOKEN s právom zápisu do repozitára; stiahnutá databáza sa pred použitím overuje a zlučuje sa iba pridávaním chýbajúcich riadkov.
- **Externé služby** – každá HTTP požiadavka má timeout; odpovede sa validujú.
- **Súborový systém** – cesty cez pathlib, relatívne voči projektu, žiadne absolútne cesty v kóde.

# 8 Použitie programu

## 8.1 Inštalácia a spustenie

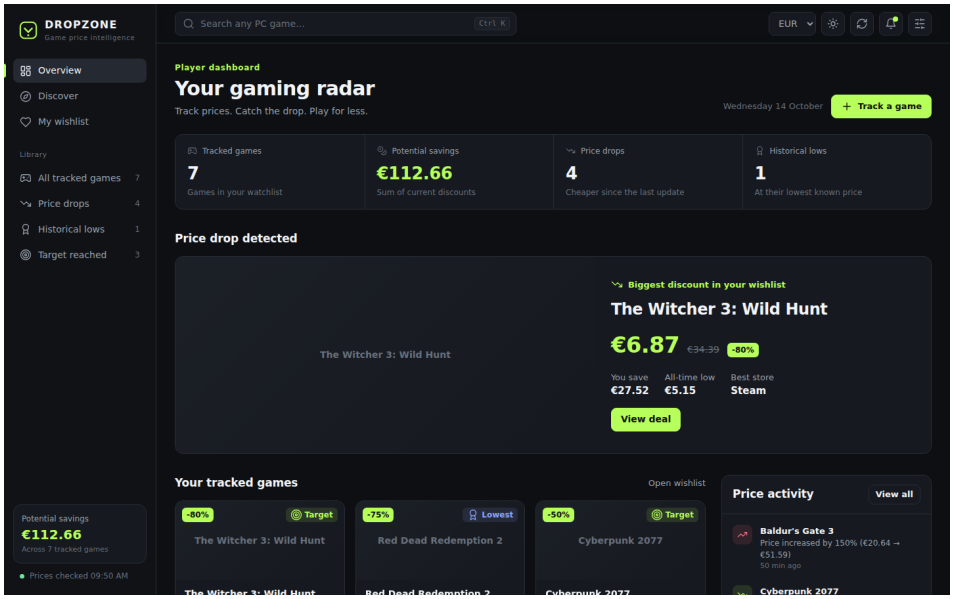
Požiadavky: Python 3.10 alebo novší, internetové pripojenie a webový prehliadač.

```
python -m venv .venv
.venv\Scripts\activate            # Linux/macOS: source .venv/bin/activate
pip install -e ".[dev]"
pytest                             # 120 passed
ruff check src tests               # All checks passed
mypy src/game_tracker              # Success: no issues found
game-tracker web                   # otvorí http://127.0.0.1:8000
```

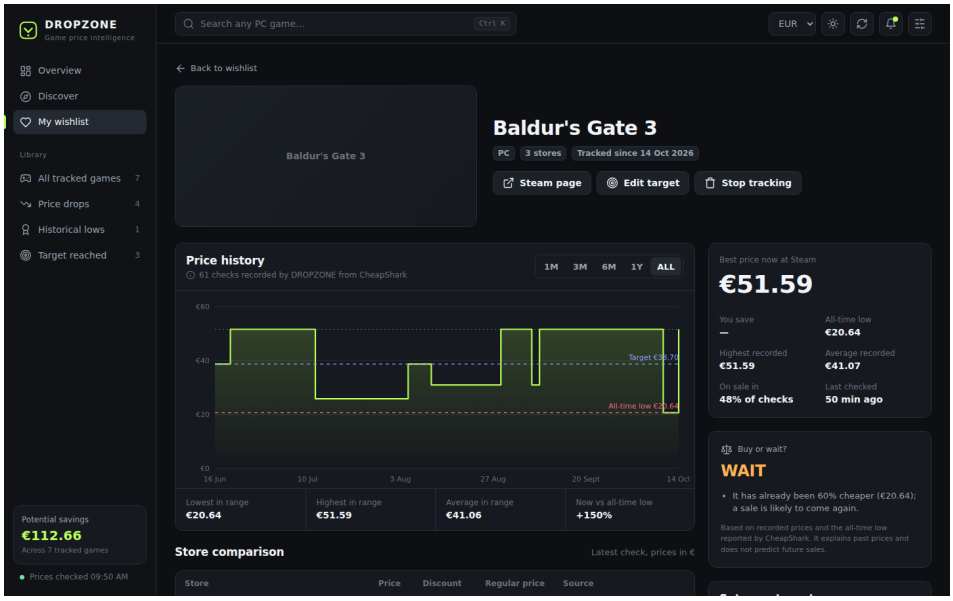
Nastavenia (port, interval kontroly cien, cesta k databáze, URL API) sú v súbore .env – vzor je v *.env.example*. Všetky hodnoty majú predvolené nastavenie, takže aplikácia funguje aj bez neho. Databáza sa vytvorí automaticky.

## 8.2 Webová aplikácia

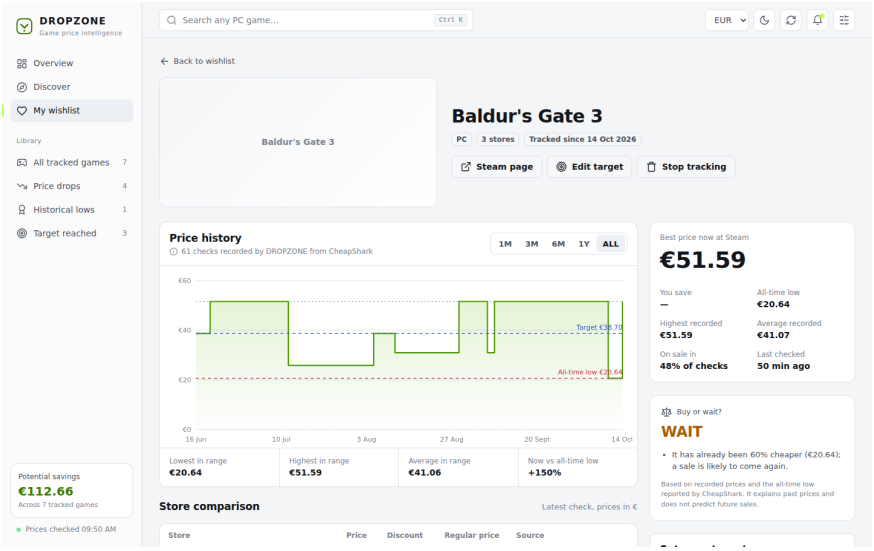
Prehľad (Overview) zobrazuje súhrnné čísla, hru s najväčšou zľavou, sledované hry a posledné zmeny cien. Hru pridáte tlačidlom *Track a game*.



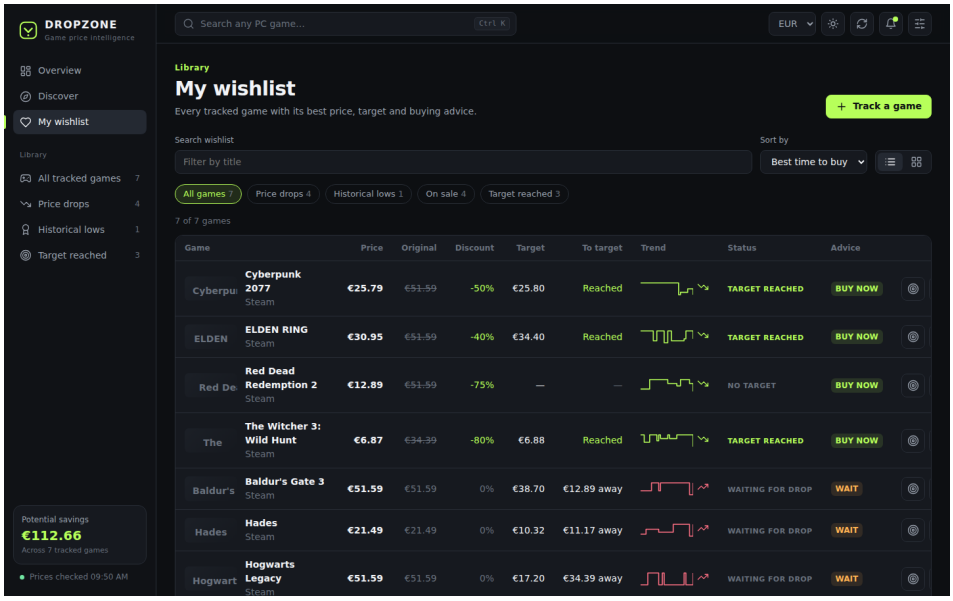

Obrázok 2: Prehľad – dashboard



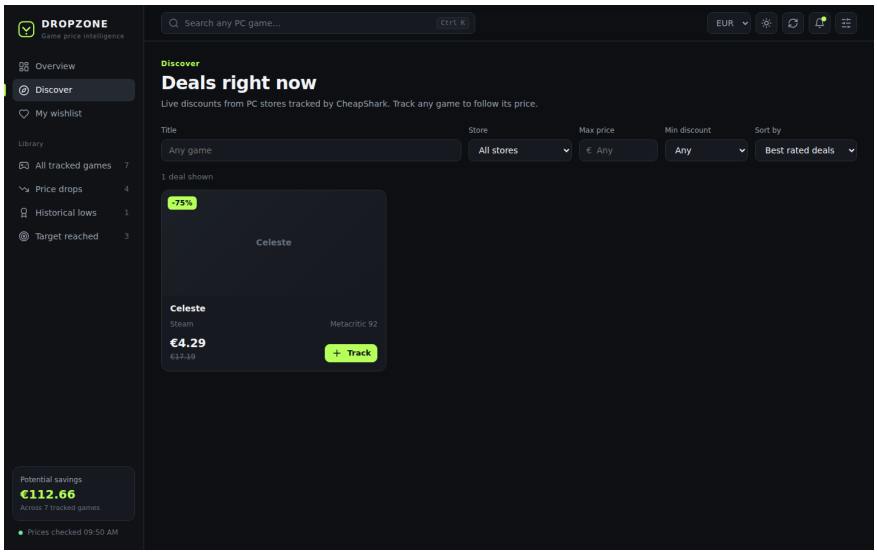

Obrázok 3: Detail hry – graf ceny, porovnanie obchodov, odporúčanie a cieľová cena




Obrázok 4: Rovnaký detail v svetlej téme (prepínač v hornej lište)

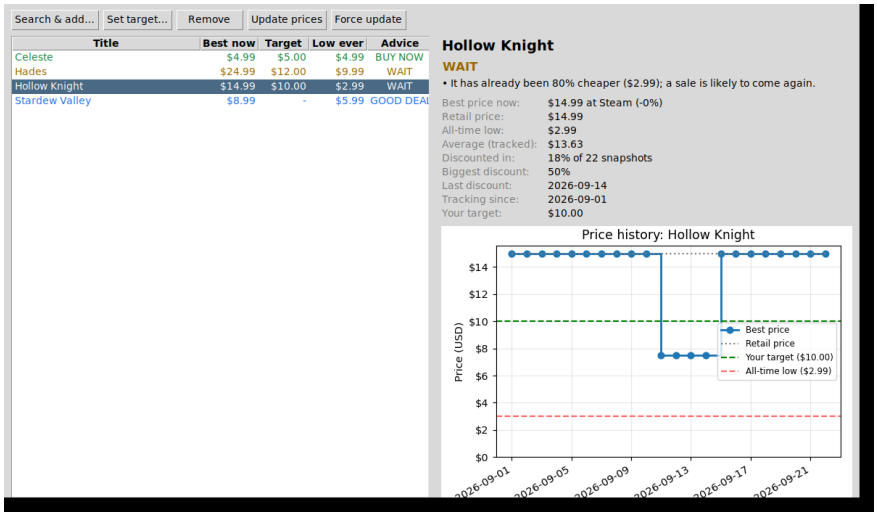



Obrázok 5: Wishlist – vyhľadávanie, filtre, triedenie a stav cieľovej ceny




Obrázok 6: Discover – aktuálne zľavy s filtrami

## 8.3 Desktopová aplikácia a CLI




Obrázok 7: Desktopová aplikácia (Tkinter)

```
game-tracker search "hollow knight"     # vyhľadanie hry a jej ID
game-tracker add 165363 --target 10     # sledovanie s cieľovou cenou (USD)
game-tracker list                       # wishlist s cenami a odporúčaním
game-tracker update                     # kontrola cien (--force ignoruje interval)
game-tracker stats 165363               # štatistiky a odporúčanie
game-tracker chart 165363 --open        # graf ako PNG
```

### 8.4 Zber cien v cloude

Jednorazové nastavenie: v súbore .env nastaviť `GT_SYNC_REPO=vlastník/repozitár`, spustiť `game-tracker sync --push` a na GitHube v záložke Actions spustiť workflow *Collect prices*. Ďalej sa ceny zbierajú automaticky a `game-tracker web` ich pri štarte zlúči s lokálnou databázou. Po pridaní alebo odstránení hier treba znova spustiť `sync --push`.

## 9 Čo som sa naučil

Najviac som sa naučil o **návrhu aplikácie vo vrstvách**. Na začiatku bol projekt iba CLI. Keď som pridával desktopové a neskôr webové rozhranie, ukázalo sa, že logika musí byť oddelená od rozhrania – inak by som rovnaký výpočet písal trikrát. Moduly *overview.py* a *service.py* vznikli práve pri tejto refaktorizácii.

Prakticky som riešil **prácu s vláknami**: Tkinter aj FastAPI spracúvajú požiadavky paralelne, pričom spojenie sqlite3 sa nesmie zdieľať medzi vláknami. Ďalej som sa naučil písať testy, ktoré nezávisia od internetu (mockovanie HTTP, vkladanie závislostí), a navrhnúť **migráciu databázy**, aby používateľ pri aktualizácii nestratil dáta.

Zaujímavým problémom bola **synchronizácia s cloudom**. Obojsmerné kopírovanie databázy by viedlo ku konfliktom, preto som určil jedného vlastníka každého údaju: zoznam hier posiela počítač (wishlist.json), nové ceny posiela cloud a lokálne sa iba pridávajú. Zlúčenie som navrhol idempotentne, aby sa dalo bezpečne spúšťať opakovane.

Zaujímavou chybou bolo, že hry pridané v prvej verzii nemali historické minimum, a keďže aktualizácia hry s čerstvou snímkou sa preskakovala, odporúčanie zostávalo nesprávne. Opravil som to a pridal regresný test.

Nabudúce by som od začiatku navrhol **API vrstvu** a spoločnú aplikačnú službu, skôr nastavil automatické spúšťanie testov (napr. GitHub Actions) a zbieranie cien by som spustil hneď v prvý deň, aby bolo na analýzu viac historických dát.

## 10 Obmedzenia a možné rozšírenia

### 10.1 Obmedzenia

- ceny sa zbierajú každých 6 hodín; GitHub môže spustenie oneskoriť o niekoľko minút a plánované workflow v repozitári bez aktivity po 60 dňoch pozastaví,
- história je krátka – pokrýva iba čas sledovania, preto sa kombinuje s minimom z CheapShark,
- CheapShark pokrýva PC obchody a ceny v USD; prepočet na EUR je orientačný (kurz ECB, nie cena obchodu),
- upozornenia sa zobrazujú iba v aplikácii (bez e-mailu alebo push notifikácií),
- aplikácia je určená pre jedného používateľa na lokálnom počítači.

### 10.2 Možné rozšírenia

- hosting webovej aplikácie na serveri s heslom, aby bola dostupná aj z mobilu,
- e-mailové alebo systémové notifikácie pri dosiahnutí cieľovej ceny,
- kolekcie hier a viacero používateľov s prihlásením,
- ceny priamo v eurách z obchodov, ktoré ich poskytujú,
- lepší model odporúčania (napr. sezónnosť výpredajov) po nazbieraní dlhšej histórie.

## 11 Zdroje a použité nástroje

- CheapShark API – dokumentácia: https://apidocs.cheapshark.com
- Frankfurter API (referenčné kurzy ECB): https://frankfurter.dev
- dokumentácie: Python (sqlite3, argparse, tkinter, concurrent.futures), httpx, pandas, matplotlib, FastAPI, Pydantic, uvicorn, pytest, ReportLab
- ikony Lucide (licencia ISC): https://lucide.dev; obrázky hier zo Steam CDN
- materiály predmetu Programovanie v Pythone: https://marek-horvath.github.io/python/

### 11.1 Použitie AI nástrojov

Pri projekte som používal AI asistenta **Claude** (Anthropic). Pomáhal mi s výberom a návrhom témy, s návrhom architektúry, s generovaním častí zdrojového kódu (backend, webové rozhranie, testy), s hľadaním chýb a s prípravou textu tejto dokumentácie. Vygenerovaný kód som spúšťal, testoval na vlastnom počítači s reálnymi dátami (pytest, ruff, mypy), prechádzal a upravoval. Rozumiem fungovaniu všetkých modulov a viem vysvetliť technické rozhodnutia opísané v tejto dokumentácii.